

AI Resume Screening & Ranking

What it does, why it works this way, and how it was built

Process & design report • generated 2026-09-23 17:20

Metric	Value
Resumes ingested	50
Eligible / rejected	41 / 9
LLM used	True
Skipped by hard gate (no LLM/GitHub)	9
GitHub enriched / no profile	6 / 25
GitHub tagged (rate-limited/error)	10 / 0
Run duration (s)	10.35

This document explains the objective, the end-to-end pipeline, the reasoning behind each design decision, and the actual results of the run over the provided resume set.

1. Objective

Build a small, production-minded backend that ingests a folder of resumes, filters candidates against a minimum Python + AI/agentive bar, scores the survivors on a 100-point rubric, enriches them with public GitHub activity, and returns a ranked shortlist in which **every score is explainable and backed by resume evidence**.

Explicit constraints respected throughout:

- A CLI is sufficient; no frontend, no auth, no database, no vector store, no deployment.
- A simple, correct, explainable backend beats an over-engineered application.
- The ranking must be reproducible and testable; hard eligibility checks stay outside the LLM.
- One bad resume, one failed model call, or one rate-limited API must not fail the batch.

2. The pipeline, end to end

Stage	What happens	Why it is here
1. Ingest	Discover files, de-duplicate by content hash, parse PDF/DOCX/TXT	Failures isolated per file so a malformed resume can't abort the batch
2. Deterministic extract	Regex + vocabulary: name, email, GitHub, skills, rough projects, evidence	Cheap, always available, and gives the LLM a validated base instead of a blank slate
3. HARD GATE (Python AND AI)	Eligibility decided on raw-text evidence only	The cheapest possible filter, run first; eliminated candidates cost nothing
4. GitHub enrichment	Only for survivors with a username; never eliminates	An additional positive signal, not an eligibility requirement
5. LLM scoring	Optional: scores each AI project's depth + evidence	Semantic judgement where it adds the most value, kept off the critical path
6. Score + rank	Combine categories, apply penalties, clamp 0-100, rank	One deterministic final number per candidate with a full rationale trail

Why this order matters

- **Gate before the LLM:** a candidate with no Python or no AI evidence is eliminated before any network call, so the model never sees candidates that cannot rank.
- **Frozen eligibility:** because eligibility is computed before the LLM, the model can neither grant nor revoke eligibility — the hard filter is provably outside the model.
- **GitHub before the LLM:** independent enrichment that overlaps with LLM pacing.

3. Eligibility — the hard gate

A candidate is **eligible only if both** hold, decided by deterministic code (*src/eligibility/rules.py*) on raw-text evidence:

- **Python evidence** — Python (or a Python-only framework such as FastAPI, Django, pandas, asyncio) appears as a real token.
- **AI/agentive evidence** — a core LLM/agentive signal, or a genuine AI project (agentive or general ML/CV/NLP) inside a project/experience section.

Positive-only, never negative

The gate asks only “is Python present?” and “is AI present?”. It never rejects *because* JavaScript, Java, React or Next.js is present — those only contribute supporting score and appear in *matched_skills*. Rejection reasons name only the unmet requirement.

Resume profile	Outcome
JavaScript / React only	Rejected — missing Python AND AI
Java / Spring Boot / React only	Rejected — missing Python AND AI
JavaScript + React + Python + AI	Eligible (JS is irrelevant to the gate)
Python + FastAPI, no AI	Rejected — missing AI

Resume profile	Outcome
AI project written in JS/TS, no Python	Rejected — Python is a hard requirement

AI evidence tiers (why a coursework mention can't rescue a profile)

Tier	Examples	Counts as AI?
1 — core LLM/agentic	LangChain, LangGraph, Google ADK, RAG, embeddings, vector search, tool calling, multi-agent, evaluation, fine-tuning, vLLM	Yes — anywhere
2 — AI ecosystem	LLM, foundation model, GenAI, Hugging Face, Bedrock, SageMaker, ASR/TTS	Yes — anywhere
3 — general ML/DS	machine learning, deep learning, CNN/RNN, XGBoost, scikit-learn, TensorFlow, PyTorch, NLP, computer vision	Only inside a project / experience section

Matching precision (false positives are the enemy)

- **Java ≠ JavaScript, SQL ≠ PostgreSQL** — enforced by word boundaries.
- **“CV” is never matched** — it means curriculum vitae on nearly every resume; only “computer vision” counts.
- “embedded systems” ≠ “embedding”; “data model” ≠ AI model; “travel agent” is not an AI agent.
- Glued and hyphenated forms tolerated: “MachineLearning”, “multi-agent”, “multi agent”.

4. Scoring — the 100-point rubric

Category	Weight	How it is earned
AI / Agentic / RAG project depth	40	best project depth (70%) + breadth of solid projects (15%) + rubric signals (15%)
Python & backend engineering	30	weighted terms × evidence-source multiplier
Cloud / deployment / full-stack	15	Docker/GCP/AWS/K8s/CI-CD; React/Next supporting
GitHub activity	10	0-5 recent activity + 0-5 maintained/relevant repositories
Engineering depth signals	5	testing, architecture, caching, queues, observability...
Project-quality penalties	−5...−15	thin LLM/API wrappers; tutorial/undetailed projects

The core idea: evidence-source weighting

Every matched term records **where** it was found, and the same term earns different credit. This directly implements the spec's requirement to reward evidence in projects/internships over keyword-only skill lists.

Where the term appears	Multiplier
Project description	1.0
Internship / work experience	0.9
Summary / preamble	0.5
Skills list only	0.4

Why these penalties

- **Thin wrapper (−5...−15)**: an “AI project” that is really a single LLM/API call with no retrieval, orchestration, tools, evaluation, state or backend workflow.
- **Tutorial / undetailed (−3)**: named like a tutorial, or described too briefly to show implementation detail or ownership.
- **Framework name in a skills list only** is capped at 0.25×40 , so it can never reach the top on naming alone.
- A classical ML project (e.g. a CNN) is **not** a thin wrapper.

5. How and why the LLM is used

- **Scope:** score each AI project's depth (0-10) and return the rubric signals, a short rationale, and a short cited-evidence snippet. It does not decide eligibility and does not extract name/email/skills (those stay deterministic).
- **Structured output:** the model is given a Pydantic response schema; the JSON is validated locally. Invalid output is discarded and the deterministic value is used.
- **Evidence required:** a score without a rationale/citation is not trusted.
- **Small prompts:** the call receives only the PROJECTS + EXPERIENCE context, not the whole resume, and returns only project scores.
- **Adapter:** one interface (*LLMClient*) with a Gemini implementation and a null fallback; swapping provider is one class. Keys come only from the environment.
- **Rate limits:** calls are paced under the tier's per-minute quota, and on a quota error the adapter waits for the reset and resumes the same request (printing a clear message) rather than silently falling back to deterministic scoring.
- **Failure isolation:** one failed call degrades a single resume and never the batch.
- **Zero-key mode:** without an API key the system still produces a full ranking via a transparent signal-based quality estimate.

6. GitHub enrichment

- Public REST API only; an optional token (from env) raises the limit from 60 to 5000 req/hr.
- Signals: recency and volume of public events (falling back to repository push dates), maintained repositories, and Python/AI-relevant repositories.
- Scoring: 0-5 activity + 0-5 repositories, hard cap 10.
- It never affects eligibility; a missing/private/rate-limited profile is a status, not a failure.
- **Tagging:** a rate-limited or errored profile is *unknown*, not inactive, so it is tagged and given a deterministic neutral placeholder (5/10) instead of 0. A genuinely absent profile still scores 0, because that is a known fact. No random values — they would break reproducibility.

7. Results of this run

Of **50** resumes, **41** were eligible and **9** were rejected. The hard gate removed **9** candidate(s) from every network call; **41** survivor(s) received LLM project scoring. GitHub was enriched for **6** candidate(s); **25** had no profile and **10** were tagged (rate-limited/error).

Shortlist — top 25 eligible candidates

#	Candidate	Total	AI	PY	Cloud	GH	Eng	Pen
1	Yash Maini	99.5	40	30	15	10	5	0
2	PRAJWAL A S	88.2	39	30	6	8	5	0
3	V Sree Raghu Vardhan	88.0	38	30	15	0	5	0
4	Hari Shanker Sharma	86.5	36	30	15	0	5	0
5	ARUNIMA SAHA	85.6	36	30	15	0	5	0
6	ABHINAV MISHRA	85.0	30	27	13	10	5	0
7	Gourab Das	85.0	30	30	15	5	5	0
8	Vivek Chimnani	83.3	29	29	15	5	5	0
9	Prathamesh Patil	82.2	35	30	15	0	5	-3
10	Sumaiya Sultana Shaik	81.7	32	30	15	0	5	0
11	Thejeshwaar Paasila	81.5	27	30	15	5	5	0
12	Kiran Naregal	78.8	36	24	13	5	4	-3
13	Nevil Jasmin Mehta	78.6	29	30	15	0	5	0
14	Aditi Kala	78.0	33	29	15	5	2	-6
15	Arjun Kumar	75.2	27	30	8	10	1	0
16	RAJ THUMAR	75.0	29	30	11	0	5	0
17	Vaibhav Wakde	74.1	32	30	7	0	5	0
18	SHUBHANSHU NAVADIYA	70.2	36	30	4	0	2	-3
19	PAVANI A	69.9	21	30	13	5	1	0
20	Jyandeep Baishya	69.7	35	25	5	0	4	0
21	ANNISHA S	69.1	26	30	7	5	1	0
22	Agam Jain	68.7	25	30	10	0	4	0
23	Vivek Shekhawat	68.0	29	24	5	5	5	0
24	PRIYANSHU KUMAR	65.8	37	23	3	0	2	0
25	HARSHA A	64.3	26	30	7	0	1	0

... and 16 more eligible candidate(s).

Why the top candidates ranked here

#1 Yash Maini — 99.5/100

Project: CopyGuard: A serverless GenAI code detection platform built on AWS Lambda and Bedrock. It provides low-latency detection for burst traffic with full infrastructure-as-code implementation.

GitHub: last public activity ~3d ago; 2 event(s) in 90d; relevant repos: face-verification-resnet50, pg-reliability-agent

- CopyGuard: depth 5.0/10 — The project demonstrates solid production-grade infrastructure and backend engineering for a GenAI service, though it lacks complex agentic features.

- Candida Hyphae Segmentation: depth 6.0/10 — The project shows significant depth in model architecture design and data pipeline optimization for medical imaging.

- NetoAI Solutions - Voice Agent: depth 8.0/10 — This project involves complex real-time orchestration and low-latency engineering for a production-grade voice agent.

#2 PRAJWAL A S — 88.2/100

Project: AI Voice Appointment Assistant: A real-time, multilingual voice AI agent for healthcare appointment management. It utilizes WebRTC for low-latency interaction and integrates with a PostgreSQL database via FastAPI to ensure grounded tool-calls.

GitHub: last public activity ~47d ago; relevant repos: voice-ai-backend, python-low-level-design, medical-ai-assistant

- AI Voice Appointment Assistant: depth 7.0/10 — The project demonstrates strong backend integration and tool-calling architecture to ensure data consistency in a real-time voice application.
- AI-Powered HPC Monitoring Assistant: depth 8.0/10 — This project features sophisticated multi-agent orchestration and complex data handling for high-performance computing environments.
- CSIR-4PI HPC Infrastructure & AI Systems: depth 10.0/10 — This is a highly mature, production-deployed system featuring rigorous evaluation, complex data pipelines, and multi-stage agentic workflows.

#3 V Sree Raghu Vardhan — 88.0/100

Project: AI-driven enterprise HRMS workflows: A complex system of multi-step AI agents integrated with enterprise APIs, semantic retrieval, and document processing to automate HR tasks. It features structured output validation, Redis-backed state management.

GitHub: No GitHub profile on resume

- AI-driven enterprise HRMS workflows: depth 10.0/10 — This is a highly sophisticated production-grade system featuring multi-step orchestration, state management, rigorous evaluation pipelines, and complex backend integration.
- Business Requirements Document generation platform: depth 5.0/10 — The project demonstrates solid backend engineering and integration of LLMs for a specific business use case, though it lacks the advanced agentic complexity.
- best-depth 28.0 + breadth 4.0 (2 solid project(s)) + signals 6.0

#4 Hari Shanker Sharma — 86.5/100

Project: Sasta Signals: A distributed price monitoring system that tracks products across 7 major Indian e-commerce platforms using automated web scraping. It features real-time price drop detection with notifications and a robust backend deployed via AWS.

GitHub: No GitHub profile on resume

- 500Agents: depth 9.0/10 — Demonstrates high-level agentic orchestration using LangGraph, multi-provider routing, and session persistence at scale.
- Jug.ai: depth 8.0/10 — Includes complex features like content training pipelines, real-time intent detection, and AI-to-human handoff logic.
- Support.cc: depth 7.0/10 — A functional business-logic-heavy application integrating RAG-like knowledge management with automated triage workflows.

#5 ARUNIMA SAHA — 85.6/100

Project: Multi-Agent Travel Planning System: An agentic travel planning and booking system that uses LangGraph to orchestrate specialized nodes for flight/hotel search and booking. It features human-in-the-loop workflows, persistent state management.

GitHub: No GitHub profile on resume

- Multi-Agent Travel Planning System: depth 9.0/10 — This is a sophisticated multi-agent system featuring complex orchestration, human-in-the-loop workflows, and robust state management.
- Graph RAG Conversational Assistant: depth 8.0/10 — The project demonstrates advanced RAG techniques using knowledge graphs and a multi-stage pipeline for query generation and result formatting.
- AI-Based Text-to-SQL Chatbot: depth 8.0/10 — This project shows strong engineering maturity through its decoupled serverless architecture, security practices, and robust logging/validation layers.

Rejected / unparsed (9) and why

Candidate	Reason(s)
AABHAR GUPTA	No evidence of Python stack; No AI/agentic project evidence
ANJALI PATIL	No evidence of Python stack; No AI/agentic project evidence
Gayathri Palraj	No AI/agentic project evidence
Harshit Kumar Mishra	No evidence of Python stack; No AI/agentic project evidence
Kartikay Sinha	No evidence of Python stack; No AI/agentic project evidence
MD AZIZUL ISLAM	No evidence of Python stack
Mukul Pal	No evidence of Python stack
SHIVAM RAJ	No evidence of Python stack; No AI/agentic project evidence
Shreya Chauhan	No AI/agentic project evidence

8. Reliability — what could go wrong, and what happens

Failure	Behaviour
Malformed / unreadable PDF	Recorded as failed; batch continues
Empty / image-only file	Recorded as empty; rejected with a reason
Duplicate file (same bytes)	De-duplicated by content hash
Two-column / glued-word PDF	Both extractors run; the higher-quality text wins

Failure	Behaviour
Missing name / email / GitHub	Safe fallbacks (email local part, filename)
LLM error or invalid JSON	Per-resume warning; deterministic score used
LLM rate limit	Waits for reset and resumes; never derails the batch
GitHub rate limit / private	Tagged + neutral placeholder; batch continues
Unexpected exception	Caught per document; batch never aborts

9. Limitations and what I would do next

Known limitations

- Rejected candidates' project names are noisy — they never reach the LLM, so their projects come only from the rough deterministic splitter (does not affect ranking).
- Two-column PDFs are largely handled, but scans/image-only resumes are not (no OCR).
- Section detection is heuristic; unusual headings may land in a catch-all bucket.
- GitHub without a token hits the 60 req/hr limit and gets tagged.
- LLM score calibration is prompt-driven (no labelled ground-truth set).

If I had more time

- Cut LLM latency: disable/minimize the model's thinking budget and cap output tokens.
- Higher throughput: a second key/model or batching several resumes per scoring call.
- Incremental runs: cache the final per-candidate result by file hash.
- One-request GraphQL GitHub instead of three REST calls.
- A labelled calibration set to tune weights and measure rank correlation vs human review.
- OCR and column-aware parsing for pathological layouts.

Eligibility is a hard, deterministic filter (Python AND AI/agentic evidence). Scores are evidence-source weighted; the LLM contributes AI project depth with cited evidence. GitHub rate-limited/errored profiles are tagged and given a neutral placeholder.